

TP 2 : Détection d'un changement ponctuel dans une suite binaire

Modèles probabilistes pour l'apprentissage - MPA

Hiba HANINI, Marwane HAJJY, Hiba FAHLI

25/09/2025

Q1. Soit $c = 1$. Donner l'expression de la loi générative $p(y \mid c = 1, \theta_1, \theta_2)$

Comme les y_i sont i.i.d :

$$p(y \mid c = 1, \theta_1, \theta_2) = \prod_{i=1}^n \theta_2^{y_i} (1 - \theta_2)^{1-y_i}$$

Q2. Même question pour $c > 1$.

$$p(y \mid c, \theta_1, \theta_2) = \prod_{i=1}^{c-1} \theta_1^{y_i} (1 - \theta_1)^{1-y_i} \prod_{i=c}^n \theta_2^{y_i} (1 - \theta_2)^{1-y_i}$$

Q3. On suppose que θ_1 et θ_2 sont connues. Dédurre des questions précédentes que

$$\forall c = 2, \dots, n, \quad \frac{p(c \mid y)}{p(c = 1 \mid y)} = \prod_{j=1}^{c-1} \frac{\theta_1^{y_j} (1 - \theta_1)^{1-y_j}}{\theta_2^{y_j} (1 - \theta_2)^{1-y_j}}.$$

Vérifier que l'on peut calculer le rapport $p(c \mid y)/p(c = 1 \mid y)$ pour tout c en effectuant de l'ordre de $n(n-1)/2$ multiplications.

On a d'après la formule de Bayes, θ_1 et θ_2 connues:

$$p(c \mid y) = p(y \mid c) \cdot p(c)$$

Et de même :

$$p(c = 1 \mid y) = p(y \mid c = 1) \cdot p(c = 1)$$

Comme $p(c = 1) = p(c) = \frac{1}{n}$

On obtient :

$$\begin{aligned}
\forall c = 2, \dots, n, \quad \frac{p(c \mid y)}{p(c = 1 \mid y)} &= \frac{p(y \mid c)}{p(y \mid c = 1)} \\
&= \frac{\prod_{i=1}^{c-1} \theta_1^{y_i} (1 - \theta_1)^{1-y_i} \prod_{i=c}^n \theta_2^{y_i} (1 - \theta_2)^{1-y_i}}{\prod_{i=1}^n \theta_2^{y_i} (1 - \theta_2)^{1-y_i}} \\
&= \prod_{j=1}^{c-1} \frac{\theta_1^{y_j} (1 - \theta_1)^{1-y_j}}{\theta_2^{y_j} (1 - \theta_2)^{1-y_j}}
\end{aligned}$$

Ainsi :

$$\boxed{\forall c = 2, \dots, n, \quad \frac{p(c \mid y)}{p(c = 1 \mid y)} = \prod_{j=1}^{c-1} \frac{\theta_1^{y_j} (1 - \theta_1)^{1-y_j}}{\theta_2^{y_j} (1 - \theta_2)^{1-y_j}}.}$$

Si on calcule chaque rapport $\frac{p(c \mid y)}{p(c = 1 \mid y)}$ indépendamment pour chaque c , alors pour un c donné, il faut effectuer $(c - 1)$ multiplications.

Le nombre total de multiplications pour tous les $c = 2, \dots, n$ est donc :

$$\boxed{\sum_{c=2}^n (c - 1) = 1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2}.}$$

D'où la vérification, on peut bien calculer tous les rapports en $\mathcal{O}\left(\frac{n(n-1)}{2}\right)$ multiplications.

Q4. Supposant θ_1 et θ_2 connues, proposer un algorithme permettant de calculer $p(c \mid y)$ pour tout $c = 1, \dots, n$ avec une complexité en $O(n)$.

Soit :

$$r_j = \frac{\theta_1^{y_j} (1 - \theta_1)^{1-y_j}}{\theta_2^{y_j} (1 - \theta_2)^{1-y_j}}.$$

Et :

$$R_c = \frac{p(c \mid y)}{p(c = 1 \mid y)} = \prod_{j=1}^{c-1} \frac{\theta_1^{y_j} (1 - \theta_1)^{1-y_j}}{\theta_2^{y_j} (1 - \theta_2)^{1-y_j}}$$

On peut alors écrire une récurrence linéaire :

$$\boxed{R_1 = 1, \quad R_c = R_{c-1} \times r_{c-1} \quad \text{pour } c = 2, \dots, n.}$$

On a par normalisation :

$$\begin{aligned}
\sum_{k=1}^n R_k &= \sum_{k=1}^n \frac{p(c = k \mid y)}{p(c = 1 \mid y)} \\
&= \frac{1}{p(c = 1 \mid y)}
\end{aligned}$$

Ainsi :

$$p(c \mid y) = p(c = 1 \mid y) \cdot R_c$$

Donc :

$$p(c | y) = \frac{R_c}{\sum_{k=1}^n R_k}$$

Algorithme en R

```
calculer_p_c <- function(y, theta1, theta2) {
  n <- length(y)
  R <- numeric(n)
  R[1] <- 1

  for (j in 1:(n - 1)) {
    r_j <- (theta1^y[j] * (1 - theta1)^(1 - y[j])) /
            (theta2^y[j] * (1 - theta2)^(1 - y[j]))
    R[j + 1] <- R[j] * r_j
  }

  somme <- sum(R)
  p <- R / somme
  return(p)
}

calculer_p_c_sans_recurrence <- function(y, theta1, theta2) {
  n <- length(y)
  R <- numeric(n)
  for (c in 1:n) {
    if (c == 1) {
      R[c] <- 1
    }
    else {
      prod_c <- 1
      for (j in 1:(c - 1)) {
        r_j <- (theta1^y[j] * (1 - theta1)^(1 - y[j])) /
                (theta2^y[j] * (1 - theta2)^(1 - y[j]))
        prod_c <- prod_c * r_j
      }
      R[c] <- prod_c
    }
  }
  p <- R / sum(R)
  return(p)
}

# Exemple :
y <- c(1, 0, 1, 1, 0)
theta1 <- 0.8
theta2 <- 0.4
p <- calculer_p_c(y, theta1, theta2)
p_sans_recurrence <- calculer_p_c_sans_recurrence(y, theta1, theta2)
print(p)
```

```
## [1] 0.13043478 0.26086957 0.08695652 0.17391304 0.34782609
```

```
print(p_sans_recurrence)
```

```
## [1] 0.13043478 0.26086957 0.08695652 0.17391304 0.34782609
```

Q5. On suppose désormais que θ_1 et θ_2 sont inconnues. À partir de valeurs initiales arbitraires, proposer un algorithme itératif, de type échantillonnage de Gibbs, permettant de calculer $p(c | y)$ pour tout $c = 1, \dots, n$ en combinant : - la simulation de la loi $p(c | y, \theta_1, \theta_2)$, - et la simulation de valeurs des paramètres θ_1 et θ_2 .

Tout comme le TP1, on utilise l'algorithme de Gibbs :

1. On initialise $\theta^{(0)} = (\theta_1^{(0)}, \theta_2^{(0)})$ et $c^{(0)}$ par des valeurs arbitraires (entre 0 et 1)
2. Pour tout $t \geq 1$, on considère l'itération du cycle suivant :
 - 2.1. Simuler $c^{(t)}$ selon $p(c | y, \theta_1^{(t-1)}, \theta_2^{(t-1)})$ (par la récurrence linéaire)
 - 2.2. Simuler θ_1 et θ_2 respectivement selon $p(\theta_1 | y, c^{(t)})$ et $p(\theta_2 | y, c^{(t)})$

Pour θ_1 , on a :

$$p(\theta_1 | y, c^{(t)}) \propto p(y | \theta_1, c^{(t)}) \cdot p(\theta_1)$$

Prenons la loi a priori non informative $\text{Unif}(0,1) = \text{Beta}(1,1)$: $\theta_1 \sim \mathcal{B} | \sqcup \neg (1,1)$

On a donc :

$$\begin{aligned} p(\theta_1 | y, c^{(t)}) &\propto p(y | \theta_1, c^{(t)}) \cdot p(\theta_1) \\ &\propto \prod_{i=1}^{c^{(t)}-1} \theta_1^{y_i} (1 - \theta_1)^{1-y_i} \\ &\propto \theta_1^{\sum_{i=1}^{c^{(t)}-1} y_i} (1 - \theta_1)^{c^{(t)}-1 - \sum_{i=1}^{c^{(t)}-1} y_i} \end{aligned}$$

Ainsi :

$$\theta_1 | y, c^{(t)} \sim \mathcal{B} | \sqcup \neg (s_1 + 1, m_1 - s_1 + 1)$$

De même pour θ_2 Posons : $\sum_{c^{(t)}} y_i = s_2$, $n - (c^{(t)} - 1) = m_2$, avec la loi à priori $\theta_2 \sim \mathcal{B} | \sqcup \neg (1,1)$

$$\theta_2 | y, c^{(t)} \sim \mathcal{B} | \sqcup \neg (s_2 + 1, m_2 - s_2 + 1)$$

Algorithme de Gibbs

```
algorithme_gibbs <- function(
  y, n_iteration, n_elimine, c_initial, theta1_initial, theta2_initial) {
  n <- length(y)

  c_val <- c_initial
  theta1 <- theta1_initial
  theta2 <- theta2_initial

  c_samples <- numeric(n_iteration) # les valeurs de c^{t}
  theta1_samples <- numeric(n_iteration)
  theta2_samples <- numeric(n_iteration)

  for (t in 1:n_iteration) {

    # Tirage de c | y, theta1, theta2
    R <- numeric(n)
    R[1] <- 1
```

```

for (j in 1:(n-1)) {
  r_j <- (theta1^y[j] * (1 - theta1)^(1 - y[j])) /
    (theta2^y[j] * (1 - theta2)^(1 - y[j]))
  R[j + 1] <- R[j] * r_j
}
somme <- sum(R)
proba_c <- R / somme
c_val <- sample(1:n, 1, prob = proba_c)

# 2. Tirage de theta1 et theta2 / y, c
s1 <- if (c_val > 1) sum(y[1:(c_val - 1)]) else 0
m1 <- c_val - 1
s2 <- sum(y[c_val:n])
m2 <- n - m1

theta1 <- rbeta(1, 1 + s1, 1 + m1 - s1)
theta2 <- rbeta(1, 1 + s2, 1 + m2 - s2)

# Sauvegarde des tirages
c_samples[t] <- c_val
theta1_samples[t] <- theta1
theta2_samples[t] <- theta2
}

# Suppression des itérations de burn-in
c_samples <- c_samples[(n_elimine + 1):n_iteration]
theta1_samples <- theta1_samples[(n_elimine + 1):n_iteration]
theta2_samples <- theta2_samples[(n_elimine + 1):n_iteration]

# Estimation de p(c | y)
p_c <- table(factor(c_samples, levels = 1:n)) / length(c_samples)

list(p_c = p_c,
     c_samples = c_samples,
     theta1_samples = theta1_samples,
     theta2_samples = theta2_samples)
}

```

Q6. Tester la convergence de votre algorithme en examinant la sensibilité aux conditions initiales choisies arbitrairement.

```

# Données observées :
y <- c(1, 0, 1, 1, 0, 0, 1, 0, 0, 1)

res1 <- algorithm_gibbs(y, n_iteration = 5000, n_elimine = 1000,
                       c_initial = 1, theta1_initial = 0.1, theta2_initial = 0.9)

res2 <- algorithm_gibbs(y, n_iteration = 5000, n_elimine = 1000,
                       c_initial = 8, theta1_initial = 0.9, theta2_initial = 0.1)

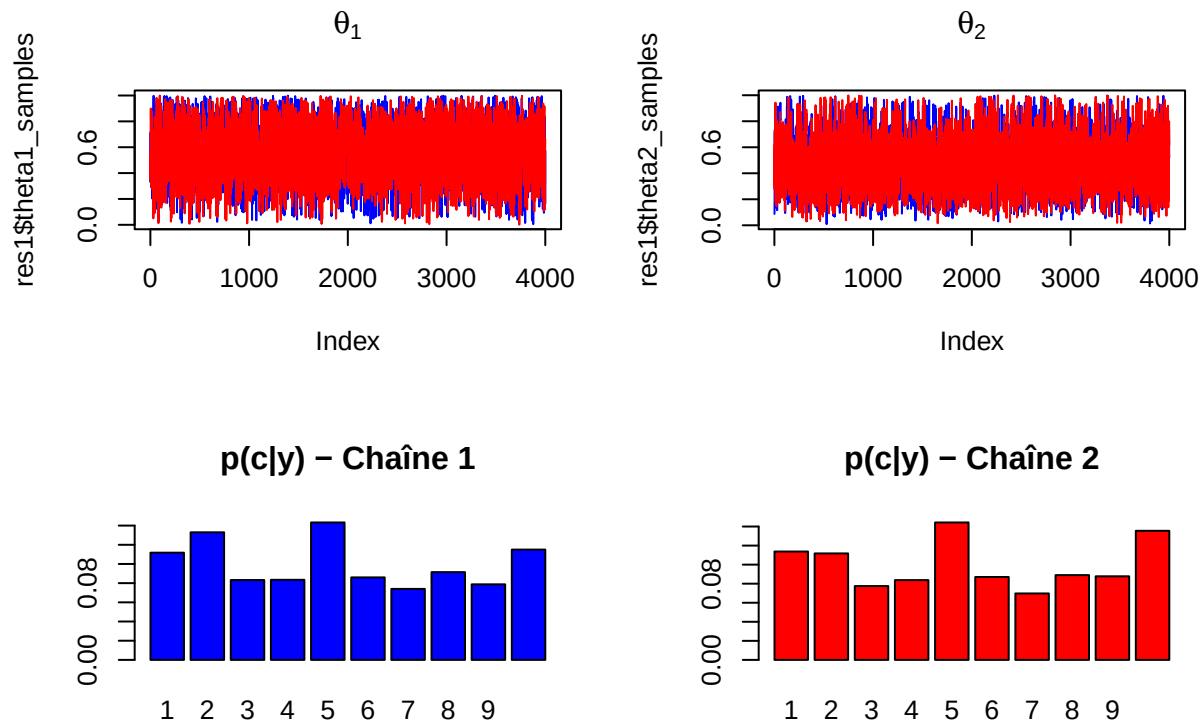
par(mfrow = c(2, 2))

plot(res1$theta1_samples, type = "l", col = "blue", main = expression(theta[1]))
lines(res2$theta1_samples, col = "red")

```

```
plot(res1$theta2_samples, type = "l", col = "blue", main = expression(theta[2]))
lines(res2$theta2_samples, col = "red")
```

```
barplot(res1$p_c, main = "p(c|y) - Chaîne 1", col = "blue")
barplot(res2$p_c, main = "p(c|y) - Chaîne 2", col = "red")
```



Q7. Analyser les jeux de données envoyés en pièce jointe par l'enseignant. Décrire l'incertitude sur le(s) point(s) de changement pour ces jeux de données (localisation des points des changements et intervalles contenant chacun des points avec une probabilité supérieure à 50%).

```
seq1 <- scan("TP2_sequence_1.txt")
seq2 <- scan("TP2_sequence_2.txt")

res1 <- algorithm_gibbs(seq1, n_iteration = 5000, n_elimine = 1000,
                        c_initial = 1, theta1_initial = 0.2, theta2_initial = 0.8)
point_estime <- which.max(as.numeric(res1$p_c))
IC <- quantile(res1$c_samples, probs = c(0.125, 0.875))
freq_avant <- mean(seq1[1:(point_estime - 1)])
freq_apres <- mean(seq1[point_estime:length(seq1)])
```

```
print("Résultats pour seq1 :")
```

```
## [1] "Résultats pour seq1 :"
```

```
print(point_estime)
```

```
## [1] 56
```

```
print(IC)
```

```
## 12.5% 87.5%
```

```
## 43 57
```

```

print(freq_avant)

## [1] 0.2545455
print(freq_apres)

## [1] 0.6533333
# Analyse pour seq2
res2 <- algorithme_gibbs(seq2, n_iteration = 5000, n_elimine = 1000,
                        c_initial = 1, theta1_initial = 0.2, theta2_initial = 0.8)

point_estime2 <- which.max(as.numeric(res2$p_c))
IC2 <- quantile(res2$c_samples, probs = c(0.125, 0.875))
freq_avant2 <- mean(seq2[1:(point_estime2 - 1)])
freq_apres2 <- mean(seq2[point_estime2:length(seq2)])

print("Résultats pour seq2 :")

## [1] "Résultats pour seq2 :"
print(point_estime2)

## [1] 315
print(IC2)

## 12.5% 87.5%
##    311    318
print(freq_avant2)

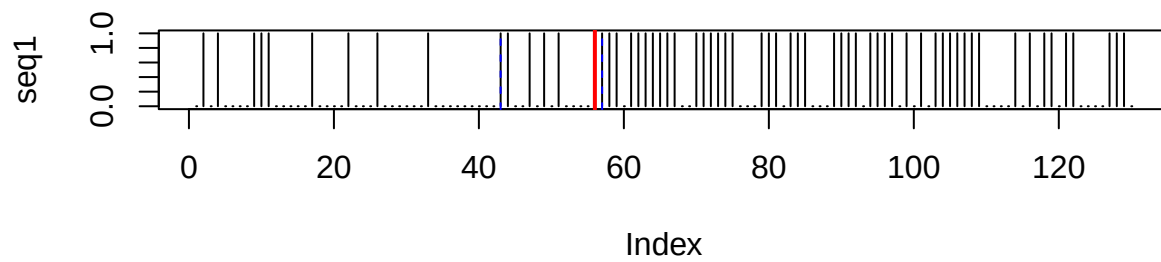
## [1] 0.3312102
print(freq_apres2)

## [1] 0.8705882
par(mfrow = c(2, 1))
plot(seq1, type = "h", main = "Seq1 avec point de changement")
abline(v = point_estime, col = "red", lwd = 2)
abline(v = IC, col = "blue", lty = 2)

plot(seq2, type = "h", main = "Seq2 avec point de changement")
abline(v = point_estime2, col = "red", lwd = 2)
abline(v = IC2, col = "blue", lty = 2)

```

Seq1 avec point de changement



Seq2 avec point de changement

